

王道考研——数据结构

WWW.CSKAOYAN.COM

第四章 串

1

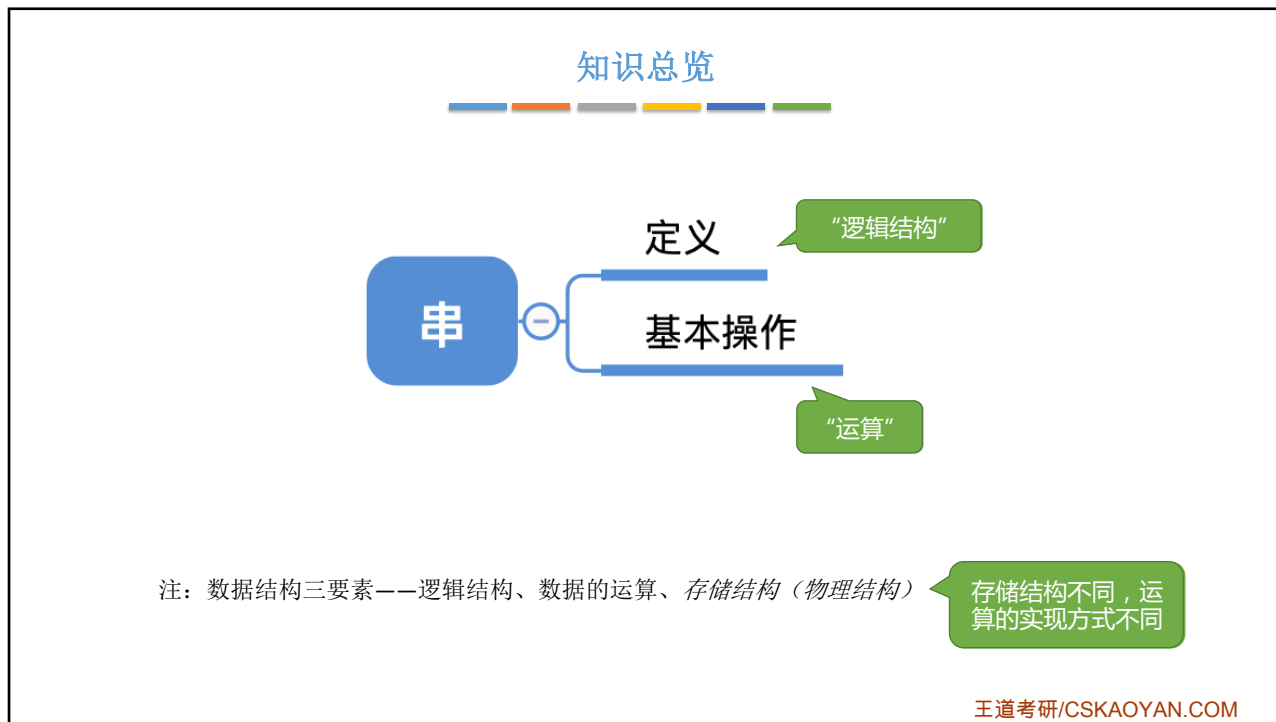
本节内容

串

定义
基本操作

王道考研/CSKAOYAN.COM

2



3

串的定义

串，即字符串（String）是由零个或多个字符组成的有限序列。一般记为 $S = 'a_1a_2 \dots a_n'$ ($n \geq 0$)

其中，S是串名，单引号括起来的字符序列是串的值； a_i 可以是字母、数字或其他字符；串中字符的个数 n 称为串的长度。 $n = 0$ 时的串称为空串（用 $\emptyset$ 表示）。

例：
 $S = \text{"HelloWorld!"}$
 $T = \text{'iPhone 11 Pro Max'}$

注：有的地方用双引号（如Java、C）
 有的地方用单引号（如Python）

子串：串中任意个连续的字符组成的子序列。
 主串：包含子串的串。
 字符在主串中的位置：字符在串中的序号。
 子串在主串中的位置：子串的第一个字符在主串中的位置。

Eg: 'iPhone', 'Pro M' 是串T的子串
 Eg: T 是子串'iPhone'的主串
 Eg: '1'在T中的位置是8(第一次出现)
 Eg: '11 Pro'在T中的位置为8

空串 V.S 空格串：
 $M = ''$ M是空串
 $N = ' '$ N是由三个空格字符组成的空格串，每个空格字符占1B

注意：位序从1开始而不是从0开始

王道考研/CSKAOYAN.COM

4

串 v.s 线性表

串是一种特殊的线性表，数据元素之间呈线性关系



串的数据对象限定为字符集（如中文字符、英文字符、数字字符、标点字符等）

串的基本操作，如增删改查等通常以子串为操作对象

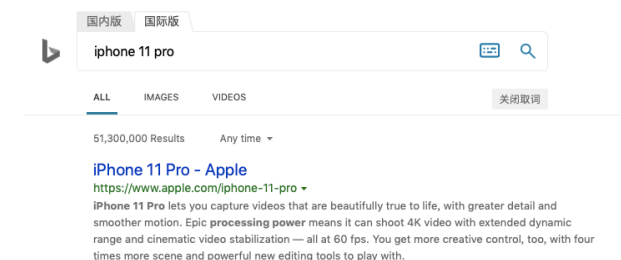


我们不一样

王道考研/CSKAOYAN.COM

5

串 v.s 线性表



通常以“子串”为增删改查的操作对象



人类的语言通常要多个字符组成的序列才有现实意义

王道考研/CSKAOYAN.COM

6

串的基本操作

假设有串T="", S="iPhone 11 Pro Max?", W="Pro"

StrAssign(&T,chars): 赋值操作。把串T赋值为chars。

StrCopy(&T,S): 复制操作。由串S复制得到串T。

StrEmpty(S): 判空操作。若S为空串, 则返回TRUE, 否则返回FALSE。

StrLength(S): 求串长。返回串S的元素个数。

ClearString(&S): 清空操作。将S清为空串。

DestroyString(&S): 销毁串。将串S销毁(回收存储空间)。

Concat(&T,S1,S2): 串联接。用T返回由S1和S2联接而成的新串。

SubString(&Sub,S,pos,len): 求子串。用Sub返回串S的第pos个字符起长度为len的子串。

Index(S,T): 定位操作。若主串S中存在与串T值相同的子串, 则返回它在主串S中第一次出现的位置; 否则函数值为0。

StrCompare(S,T): 比较操作。若S>T, 则返回值>0; 若S=T, 则返回值=0; 若S<T, 则返回值<0。

Eg: 执行基本操作 Concat(&T, S, W) 后, T="iPhone 11 Pro Max?Pro"

执行基本操作 SubString(&T,S, 4, 6)后, T="one 11"

执行基本操作 Index(S, W)后, 返回值为 11

存储空间扩展?

王道考研/CSKAOYAN.COM

7

串的比较操作

StrCompare(S,T): 比较操作。若S>T, 则返回值>0; 若S=T, 则返回值=0; 若S<T, 则返回值<0。

A

abandon/ ə'bəndən/ vt. 丢弃; 放弃, 抛弃

aboard/ ə'bo:d/ ad. 在船(车)上; 上船

absolute/ 'æbsəlu:t/ a. 绝对的; 纯粹的

absolutely/ 'æbsəlu:tli/ ad. 完全地; 绝对地

absorb/ əb'sɔ:b/ vt. 吸收; 使专心

abstract/ 'æbstrækt/ n. 摘要

abundant/ ə'bʌndənt/ a. 丰富的; 大量的

abuse/ ə'bjʊ:z, ə'bjʊ:s/ vt. 滥用; 虐待 n. 滥用

academic/ æke'demik/ a. 学院的; 学术的

accelerate/ æk'seləreit/ vt. (使)加快; 促进

"abandon" < "aboard"

从第一个字符开始往后依次对比,
先出现更大字符的串就更大

长串的前缀与短串相
同时, 长串更大

"abstract" < "abstraction"

"abstract" < "abstract"

只有两个串完全相
同时, 才相等

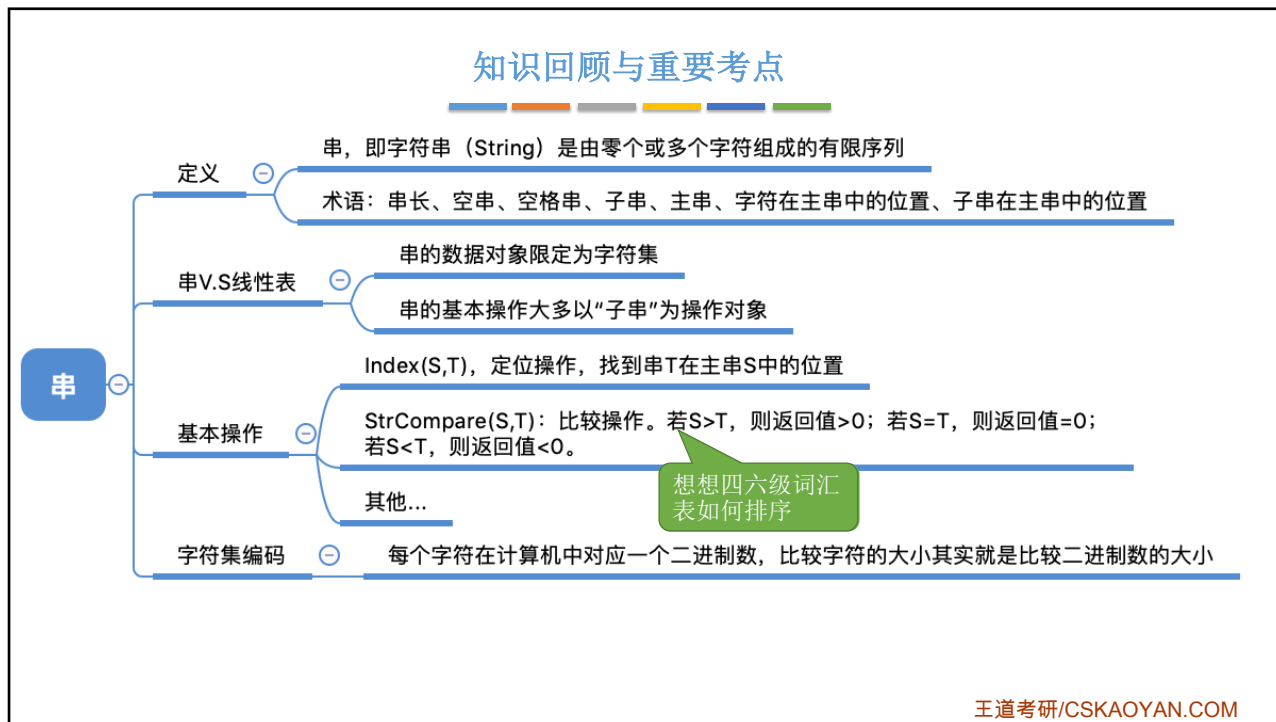
"academic" > "abuse"

"academic" = "academic"

四级词汇表(正序版)

王道考研/CSKAOYAN.COM

8



本节内容

串
存储结构

王道考研/CSKAOYAN.COM

1

知识总览

串
的
存
储
结
构

顺序存储

链式存储

基于顺序存储实现基本操作

王道考研/CSKAOYAN.COM

2

串的顺序存储

结合顺序表的知识思考优缺点

```

#define MAXLEN 255 //预定义最大串长为255
typedef struct{
    char ch[MAXLEN]; //每个分量存储一个字符
    int length; //串的实际长度
}SString;

typedef struct{
    char *ch; //按串长分配存储区, ch指向串的基地址
    int length; //串的长度
}HString;
HString S;
S.ch = (char *) malloc(MAXLEN * sizeof(char));
S.length = 0;
    
```

静态数组实现 (定长顺序存储)

动态数组实现 (堆分配存储)

分配连续的存储空间, 每个 char 字符占 1B

串的实际长度

用完需要手动free

内存

S.ch →

ch[0]
ch[1]
ch[2]
ch[3]
ch[4]
...
ch[254]
length

王道考研/CSKAOYAN.COM

3

串的顺序存储

char ch[10]

方案一:

方案二:

方案三:

方案四: (教材)

ch[0]充当 Length

变量Length

没有Length变量, 以字符'\0'表示结尾 (对应ASCII码的0)

ch[0]废弃不用

变量Length

王道考研/CSKAOYAN.COM

4

串的链式存储

```
typedef struct StringNode{
    char ch;    //每个结点存1个字符
    struct StringNode * next;
}StringNode, * String;
```

S → a → b → c → d → e → NULL

```
typedef struct StringNode{
    char ch[4];    //每个结点存多个字符
    struct StringNode * next;
}StringNode, * String;
```

S → a b c d → e # # # → NULL

结合链表的知识
思考优缺点

存储密度低: 每个字
符1B, 每个指针4B

存储密度
提高

王道考研/CSKAOYAN.COM

5

基本操作的实现

ch[0]废
弃不用

方案四:
(教材)

```
#define MAXLEN 255    //预定义最大串长为255
typedef struct{
    char ch[MAXLEN];    //每个分量存储一个字符
    int length;    //串的实际长度
}SString;
```

变量Length

StrAssign(&T,chars): 赋值操作。把串T赋值为chars。
 StrCopy(&T,S): 复制操作。由串S复制得到串T。
 StrEmpty(S): 判空操作。若S为空串, 则返回TRUE, 否则返回FALSE。
 StrLength(S): 求串长。返回串S的元素个数。
 ClearString(&S): 清空操作。将S清为空串。
 DestroyString(&S): 销毁串。将串S销毁(回收存储空间)。
 Concat(&T,S1,S2): 串联接。用T返回由S1和S2联接而成的新串

王道考研/CSKAOYAN.COM

6

基本操作的实现

SubString(&Sub,S,pos,len): 求子串。用Sub返回串S的第pos个字符起长度为len的子串。

S.ch="wangdao"
S.length=7

S		w	a	n	g	d	a	o		
	0	1	2	3	4	5	6	7	8	9

```
//求子串
bool SubString(SString &Sub, SString S, int pos,int len){
    //子串范围越界
    if (pos+len-1 > S.length)
        return false;
    for (int i=pos; i<pos+len; i++)
        Sub.ch[i-pos+1] = S.ch[i];
    Sub.length = len;
    return true;
}

#define MAXLEN 255 //预定义最大串长为255
typedef struct{
    char ch[MAXLEN]; //每个分量存储一个字符
    int length; //串的实际长度
}SString;
```

王道考研/CSKAOYAN.COM

7

基本操作的实现

StrCompare(S,T): 比较操作。若S>T, 则返回值>0; 若S=T, 则返回值=0; 若S<T, 则返回值<0。

S.ch="wangdao"
S.length=7

S.ch		w	a	n	g	d	a	o		
	0	1	2	3	4	5	6	7	8	9

//比较操作。若S>T, 则返回值>0; 若S=T, 则返回值=0; 若S<T, 则返回值<0

```
int StrCompare(SString S, SString T) {
    for (int i=1; i<=S.length && i<=T.length; i++){
        if (S.ch[i]!=T.ch[i])
            return S.ch[i]-T.ch[i];
    }
```

//扫描过的所有字符都相同, 则长度长的串更大

```
return S.length-T.length;
}
```

T1		w	b	n	g	d	a	o		
	0	1	2	3	4	5	6	7	8	9
T2		w	a	n	g					
	0	1	2	3	4	5	6	7	8	9

王道考研/CSKAOYAN.COM

8

基本操作的实现

Index(S,T): 定位操作。若主串S中存在与串T值相同的子串, 则返回它的主串S中第一次出现的位置; 否则函数值为0。

S.ch="wangdao"
S.length=7

S.ch	→		w	a	n	g	d	a	o		
		0	1	2	3	4	5	6	7	8	9

```
int Index(SString S, SString T){
    int i=1, n=StrLength(S), m=StrLength(T);
    SString sub;    //用于暂存子串
    while(i<=n-m+1){
        SubString(sub,S,i,m);
        if(StrCompare(sub, T)!=0) ++i;
        else return i;    //返回子串在主串中的位置
    }
    return 0;    //S中不存在与T相等的子串
}
```

T		g	d	a						
	0	1	2	3	4	5	6	7	8	9

王道考研/CSKAOYAN.COM

9

知识回顾与重要考点

```
#define MAXLEN 255    //预定义最大串长为255
typedef struct{
    char ch[MAXLEN];    //每个分量存储一个字符
    int length;    //串的实际长度
}SString;
```

串的存储结构



王道考研/CSKAOYAN.COM

10

本节内容

KMP算法

朴素模式匹配算法
的优化

王道考研/CSKAOYAN.COM

1

KMP算法

由D.E.Knuth, J.H.Morris和V.R.Pratt提出, 因此称为 KMP算法



你不要凶我 我害怕。

王道考研/CSKAOYAN.COM

2

最坏情况

Diagram 1: Initial matching. S: [a][a][a][a][a][a][a][a][b], T: [a][a][a][b]. i=1, j=1. Match at (1,1).

Diagram 2: i=2, j=2. Match at (2,2).

Diagram 3: i=3, j=3. Match at (3,3).

Diagram 4: i=4, j=4. Mismatch at (4,4). Callout: 主串的扫描指针 i: 前进m步, 后退m-1步, 再前进m步, 再后退m-1步...

若模式串长度为m, 主串长度为n, 则直到匹配成功/匹配失败最多需要 $n*m$ 次比较
 最坏时间复杂度: $O(nm)$

王道考研/CSKAOYAN.COM

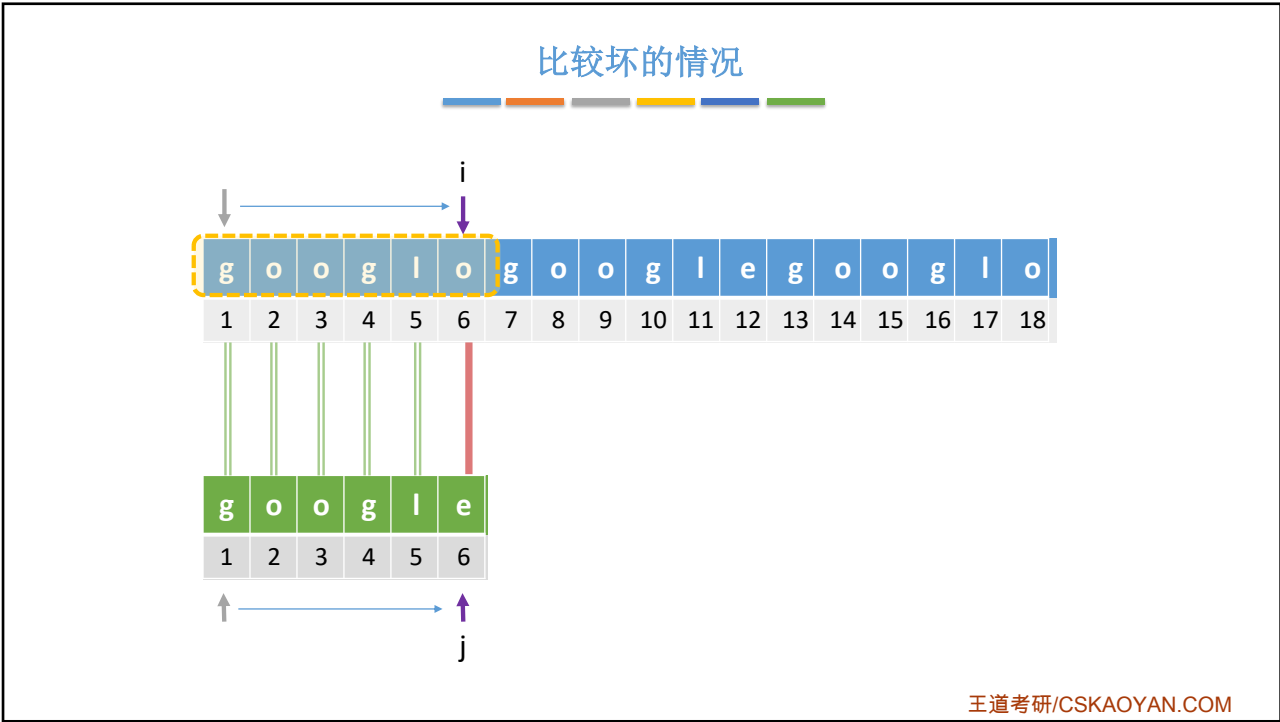
3

比较坏的情况

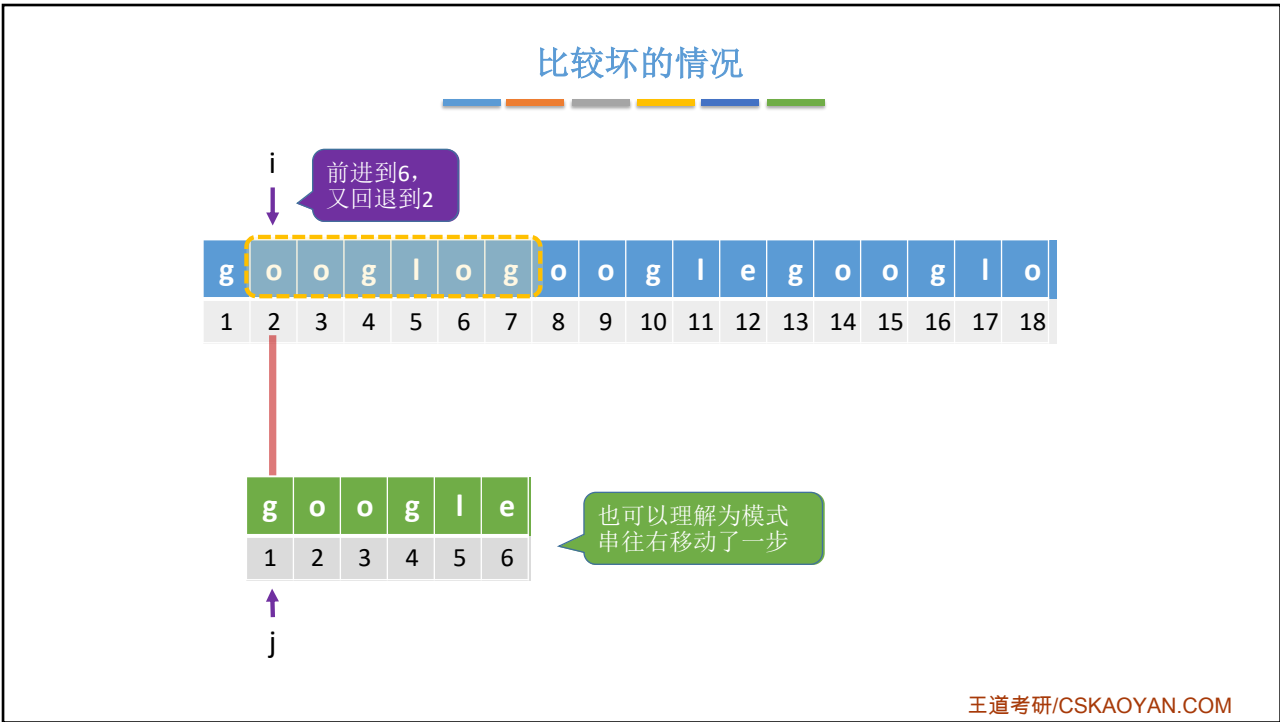
GoogloGooglo

王道考研/CSKAOYAN.COM

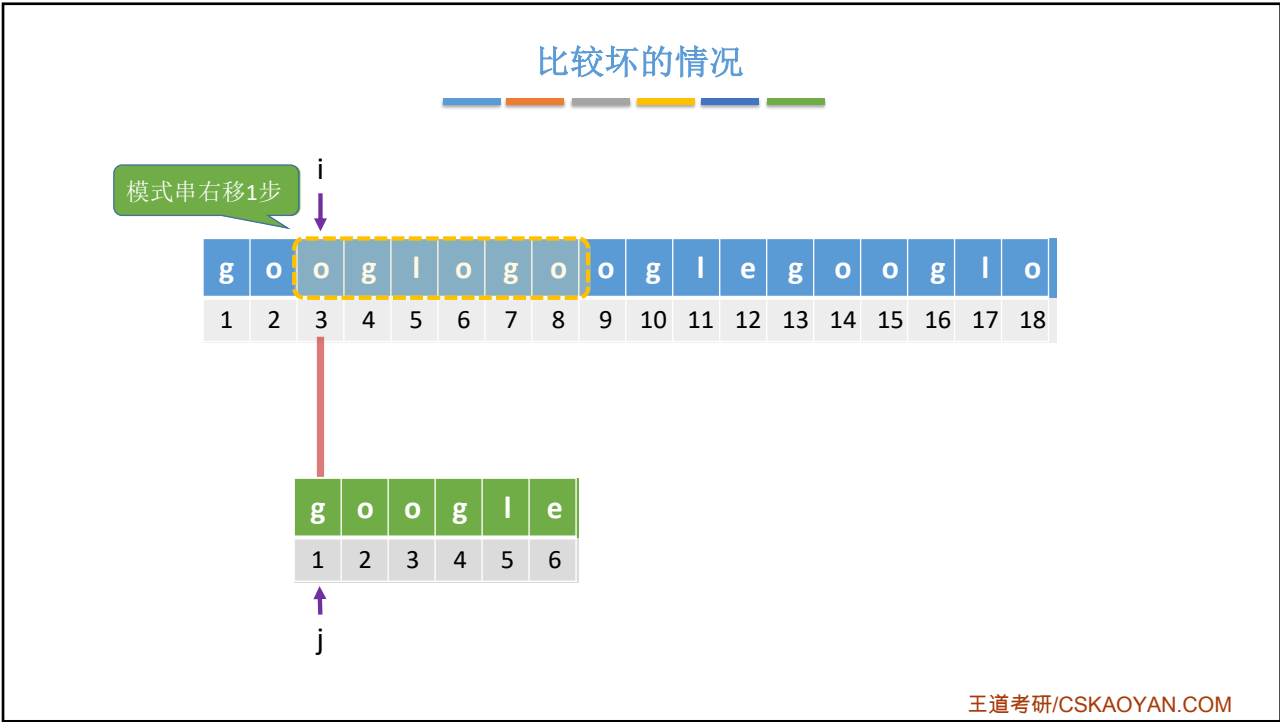
4



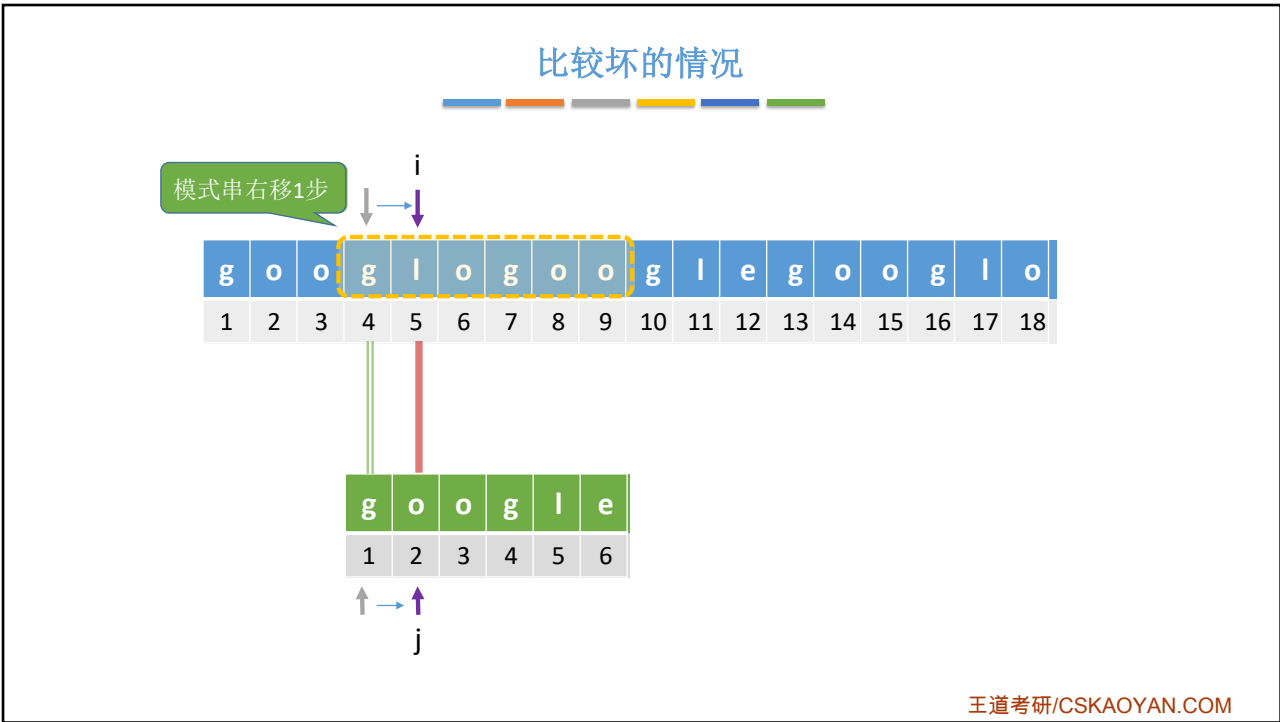
5



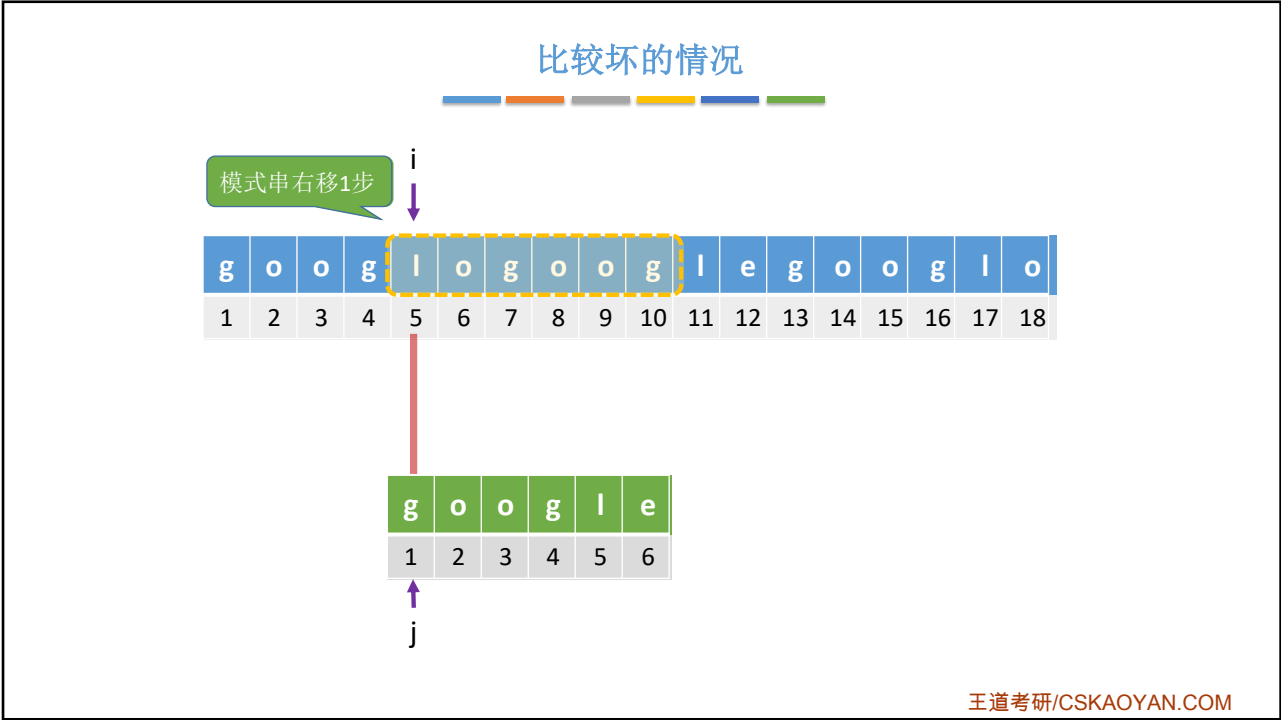
6



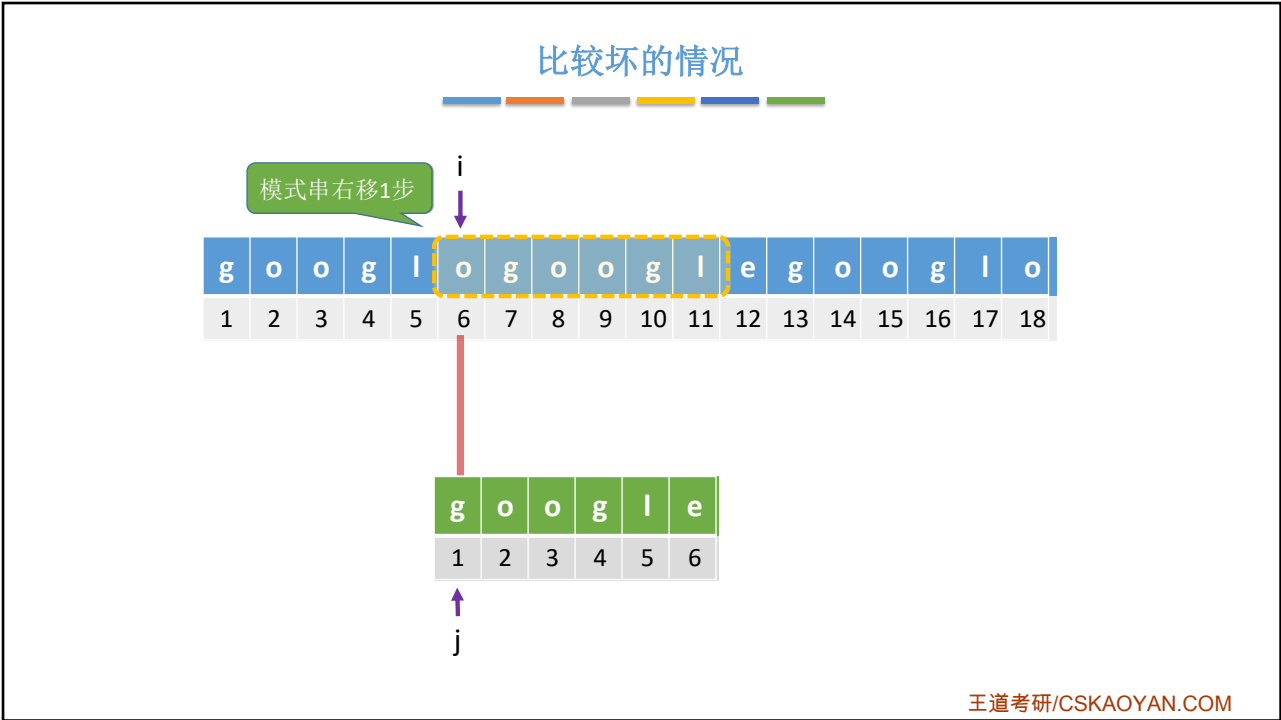
7



8



9



10

比较坏的情况

模式串右移1步

g o o g l o g o o g l e g o o g l o

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18

g o o g l e

1 2 3 4 5 6

i

j

朴素模式匹配算法的缺点：
当某些子串与模式串能部分匹配时，主串的扫描指针i经常回溯，导致时间开销增加

王道考研/CSKAOYAN.COM

11

看看人脑的反应

马赛克

任务：找到'google'

要求：你的目光不要回溯

你的目光相当于指针i

王道考研/CSKAOYAN.COM

12

王道考研,cskaoyan.com

6

解决主串回溯问题

这个字符不是e,
你会怎么做?

i

googl?

马赛克

值得期待,
是否是 g

j

任务: 找到'google'

↑

←

↑

改进思路: 主串指针不回溯,
只有模式串指针回溯

王道考研/CSKAOYAN.COM

13

模式串指针应该回溯到哪儿?

这个字符不是'e',
该怎么做?

i

值得期待
是否是 g

如果 j = k 时才发现匹配失败,
说明 1~k-1 都匹配成功

x	x	x	x	x	g	o	o	g	l	?	?	?	?	?	?	?	?
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

g o | o | g | l |

e

g	o	o	g	l	e
1	2	3	4	5	6

↑

←

↑

若 j=6 时发生不匹配, 则应让 j 回到 1

王道考研/CSKAOYAN.COM

14

模式串指针应该回溯到哪儿？

主串的指针没有回溯

值得期待 是否是 g

x	x	x	x	x	g	o	o	g	l	?	?	?	?	?	?	?	?
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

少了5次对比

若 j=6 时发生不匹配，则应让 j 回到 1

也可以理解为模式串往右移动了5步

g	o	o	g	l	e
1	2	3	4	5	6

j

王道考研/CSKAOYAN.COM

15

模式串指针应该回溯到哪儿？

这个字符是 'g'，该怎么做？

应该让 i++，j++

x	x	x	x	x	g	o	o	g	l	g	?	?	?	?	?	?	?
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

g	o	o	g	l	e
1	2	3	4	5	6

j

王道考研/CSKAOYAN.COM

16

模式串指针应该回溯到哪儿?

这个字符不是'g', 该怎么做?

应该让 i 后移, 检查下一个字符是否是g

若 j=1 时发生不匹配, 则应让 i++, 而 j 依然是 1

王道考研/CSKAOYAN.COM

17

模式串指针应该回溯到哪儿?

这个字符不是'l', 该怎么做?

值得期待 是否是 o

如果 j = k 时才发现匹配失败, 说明 1~k-1 都匹配成功

若 j=5 时发生不匹配, 则应让 j 回到 2

王道考研/CSKAOYAN.COM

18

模式串指针应该回溯到哪儿？

这个字符不是'l'，
该怎么做？

值得期待
是否是o

如果 $j = k$ 时才发现匹配失败，
说明 $1 \sim k-1$ 都匹配成功

x	x	x	x	g	o	o	g	?	?	?	?	?	x	x	x	x	x
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

少了3次对比

若 $j=5$ 时发生不匹配，则应让 j 回到 2

g	o	o	g	l	e
1	2	3	4	5	6

j

王道考研/CSKAOYAN.COM

19

模式串指针应该回溯到哪儿？

这个字符不是'g'，
该怎么做？

值得期待
是否是g

如果 $j = k$ 时才发现匹配失败，
说明 $1 \sim k-1$ 都匹配成功

x	x	g	o	o	?	?	?	?	?	?	?	?	x	x	x	x	x
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

若 $j=4$ 时发生不匹配，则应让 j 回到 1

g	o	o	g	l	e
1	2	3	4	5	6

j

王道考研/CSKAOYAN.COM

20

模式串指针应该回溯到哪儿？

这个字符不是'g'，
该怎么做？

值得期待
是否是 g

如果 $j = k$ 时才发现匹配失败，
说明 $1 \sim k-1$ 都匹配成功

x	x	g	o	o	?	?	?	?	?	?	?	?	x	x	x	x	x
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

少了2次对比

g	o	o	g	l	e
1	2	3	4	5	6

若 $j=4$ 时发生不匹配，则应让 j 回到 1

王道考研/CSKAOYAN.COM

21

模式串指针应该回溯到哪儿？

这个字符不是'o'，
该怎么做？

值得期待
是否是 g

如果 $j = k$ 时才发现匹配失败，
说明 $1 \sim k-1$ 都匹配成功

x	x	g	o	?	?	?	?	?	?	?	?	?	x	x	x	x	x
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

g	o	o	g	l	e
1	2	3	4	5	6

若 $j=3$ 时发生不匹配，则应让 j 回到 1

王道考研/CSKAOYAN.COM

22

模式串指针应该回溯到哪儿？

这个字符不是'o'，该怎么做？

值得期待是否是g

如果 $j = k$ 时才发现匹配失败，说明 $1 \sim k-1$ 都匹配成功

x	x	g	o	?	?	?	?	?	?	?	?	?	x	x	x	x	x
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

少了1次对比

g	o	o	g	l	e
1	2	3	4	5	6

j

若 $j=3$ 时发生不匹配，则应让 j 回到 1

王道考研/CSKAOYAN.COM

23

模式串指针应该回溯到哪儿？

这个字符不是'o'，该怎么做？

值得期待是否是g

如果 $j = k$ 时才发现匹配失败，说明 $1 \sim k-1$ 都匹配成功

x	x	g	?	?	?	?	?	?	?	?	?	?	x	x	x	x	x
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

g	o	o	g	l	e
1	2	3	4	5	6

j

若 $j=2$ 时发生不匹配，则应让 j 回到 1

王道考研/CSKAOYAN.COM

24

模式串指针应该回溯到哪儿?

这个字符不是'o', 该怎么做?

i

↓

x	x	g	?	?	?	?	?	?	?	?	?	?	x	x	x	x	x
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

值得期待 是否是 g

如果 $j = k$ 时才发现匹配失败, 说明 $1 \sim k-1$ 都匹配成功

g	o	o	g	l	e
1	2	3	4	5	6

↑

j

若 $j=2$ 时发生不匹配, 则应让 j 回到 1

王道考研/CSKAOYAN.COM

25

模式串指针应该回溯到哪儿?

如果 $j = k$ 时才发现匹配失败, 说明 $1 \sim k-1$ 都匹配成功

g	o	o	g	l	e
1	2	3	4	5	6

↑

j

int next[7];

0	1	2	3	4	5	6
	0	1	1	1	2	1

当 $j=k$ 且发现字符不匹配时, 令 $j=next[k]$

若当前两个字符匹配, 则 $i++$, $j++$

若 $j=1$ 时发生不匹配, 则应让 $i++$, 而 j 依然是 1

若 $j=2$ 时发生不匹配, 则应让 j 回到 1

若 $j=3$ 时发生不匹配, 则应让 j 回到 1

若 $j=4$ 时发生不匹配, 则应让 j 回到 1

若 $j=5$ 时发生不匹配, 则应让 j 回到 2

若 $j=6$ 时发生不匹配, 则应让 j 回到 1

王道考研/CSKAOYAN.COM

26

KMP算法代码

```
int Index_KMP(SString S, SString T, int next[]){
    int i=1, j=1;
    while(i<=S.length&& j<=T.length){
        if(j==0 || S.ch[i]==T.ch[j]){
            ++i;
            ++j;           //继续比较后继字符
        }
        else {
            j=next[j];     //模式串向右移动
        }
        if(j>T.length)
            return i-T.length; //匹配成功
        else
            return 0;
    }
}
```

g	o	o	g	l	e
1	2	3	4	5	6

↑
j

int next[7];

0	1	2	3	4	5	6
	0	1	1	1	2	1

当 j=k 且发现字符不匹配时, 令 j=next[k]

王道考研/CSKAOYAN.COM

本节内容

KMP算法

求next数组

王道考研/CSKAOYAN.COM

1

KMP算法代码

```

int Index_KMP(SSString S,SSString T,int next[]){
    int i=1, j=1;
    while(i<=S.length&& j<=T.length){
        if(j==0 || S.ch[i]==T.ch[j]){
            ++i;
            ++j;           //继续比较后继字符
        }
        else {
            j=next[j];     //模式串向右移动
        }
    }
    if(j>T.length)
        return i-T.length; //匹配成功
    else
        return 0;
}
            
```

g	o	o	g	l	e
1	2	3	4	5	6

↑
j

由模式串确定的next数组

```
int next[7];
```

0	1	2	3	4	5	6
0	1	1	1	2	1	

当 j=k 且发现字符不匹配时, 令j=next[k]

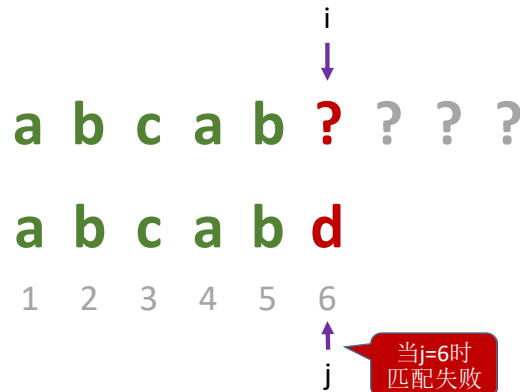
王道考研/CSKAOYAN.COM

2

观察: 求模式串的next数组

next 数组: 当模式串的第 j 个字符匹配失败时, 令模式串跳到 $\text{next}[j]$ 再继续匹配

模式串: 'abcabd'



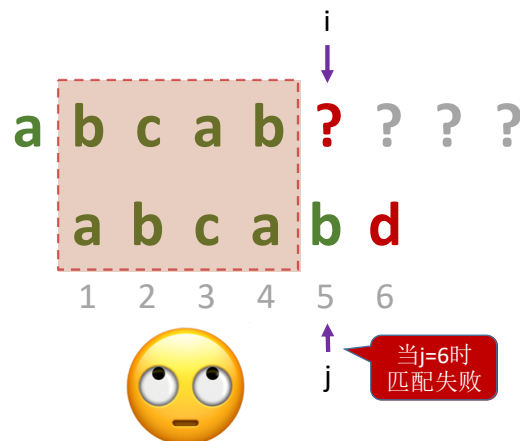
王道考研/CSKAOYAN.COM

3

观察: 求模式串的next数组

next 数组: 当模式串的第 j 个字符匹配失败时, 令模式串跳到 $\text{next}[j]$ 再继续匹配

模式串: 'abcabd'



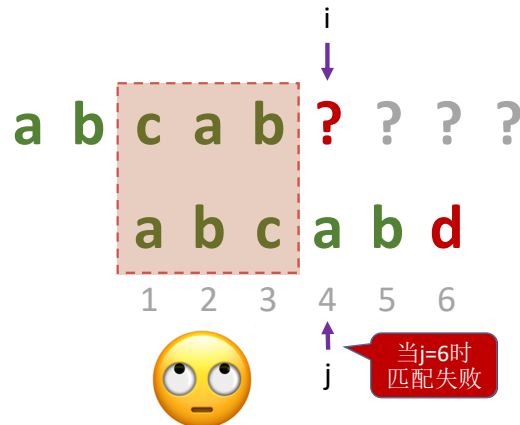
王道考研/CSKAOYAN.COM

4

观察: 求模式串的next数组

next 数组: 当模式串的第 j 个字符匹配失败时, 令模式串跳到 $\text{next}[j]$ 再继续匹配

模式串: 'abcabd'



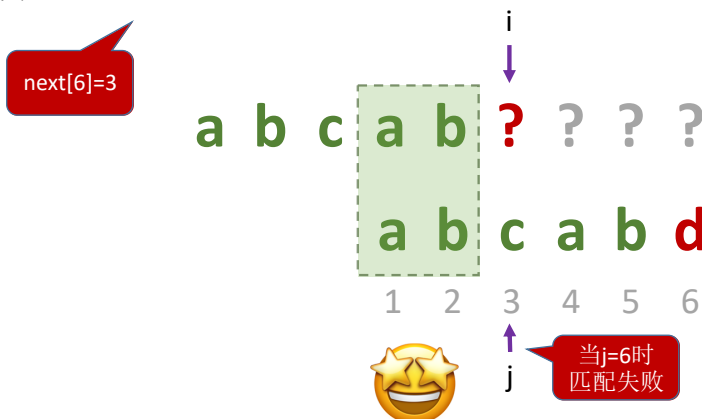
王道考研/CSKAOYAN.COM

5

观察: 求模式串的next数组

next 数组: 当模式串的第 j 个字符匹配失败时, 令模式串跳到 $\text{next}[j]$ 再继续匹配

模式串: 'abcabd'



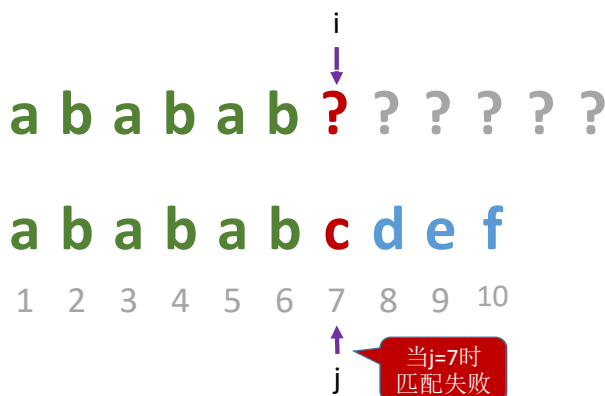
王道考研/CSKAOYAN.COM

6

观察: 求模式串的next数组

next 数组: 当模式串的第 j 个字符匹配失败时, 令模式串跳到 $\text{next}[j]$ 再继续匹配

模式串: 'abababcdef'



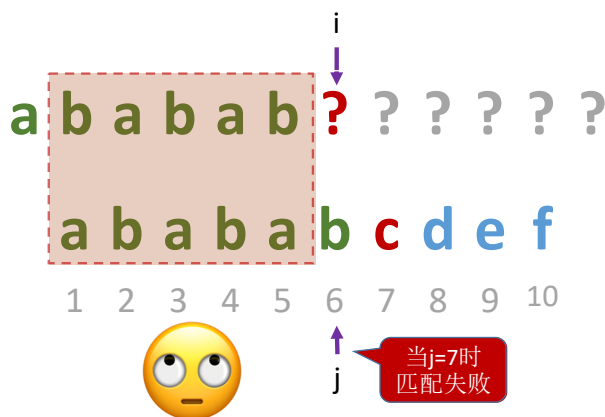
王道考研/CSKAOYAN.COM

7

观察: 求模式串的next数组

next 数组: 当模式串的第 j 个字符匹配失败时, 令模式串跳到 $\text{next}[j]$ 再继续匹配

模式串: 'abababcdef'



王道考研/CSKAOYAN.COM

8

观察：求模式串的next数组

next 数组：当模式串的第 j 个字符匹配失败时，令模式串跳到 next[j] 再继续匹配

模式串：'abababcdef'

next[7]=5

a b a b a b ? ? ? ? ? ?

a b a b a b c d e f

1 2 3 4 5 6 7 8 9 10

😊

i

j

当j=7时
匹配失败

王道考研/CSKAOYAN.COM

9

观察：求模式串的next数组

next 数组：当模式串的第 j 个字符匹配失败时，令模式串跳到 next[j] 再继续匹配

模式串：'abababcdef'

next[7]=3?
可以不?

a b a b a b ? ? ? ? ? ?

a b a b a b c d e f

1 2 3 4 5 6 7 8 9 10

j

当j=7时
匹配失败

王道考研/CSKAOYAN.COM

10

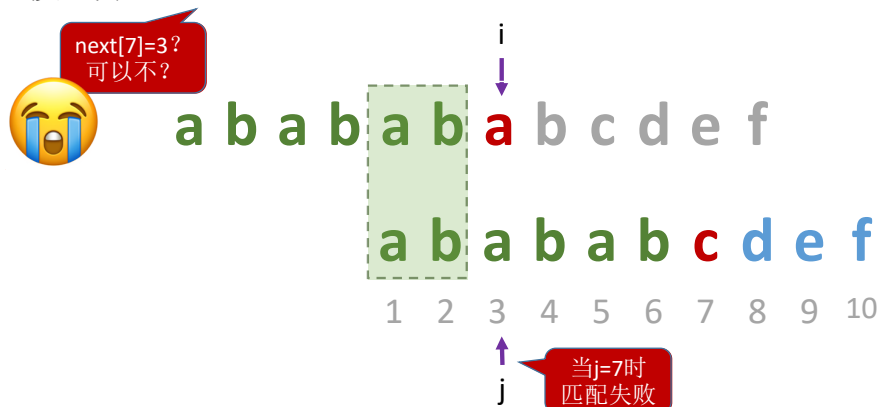
王道考研/CSKAOYAN.COM

5

观察: 求模式串的next数组

next 数组: 当模式串的第 j 个字符匹配失败时, 令模式串跳到 $\text{next}[j]$ 再继续匹配

模式串: 'abababcdef'



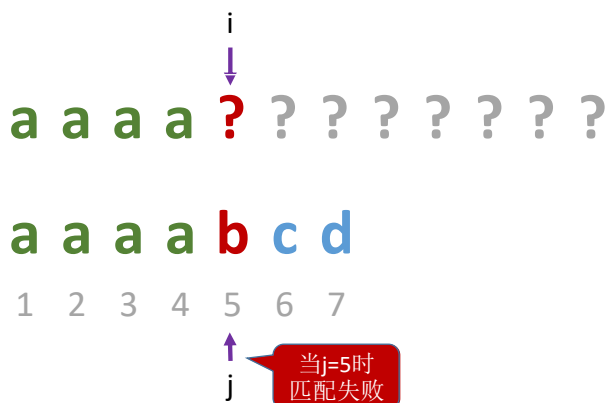
王道考研/CSKAOYAN.COM

11

观察: 求模式串的next数组

next 数组: 当模式串的第 j 个字符匹配失败时, 令模式串跳到 $\text{next}[j]$ 再继续匹配

模式串: 'aaaabcd'



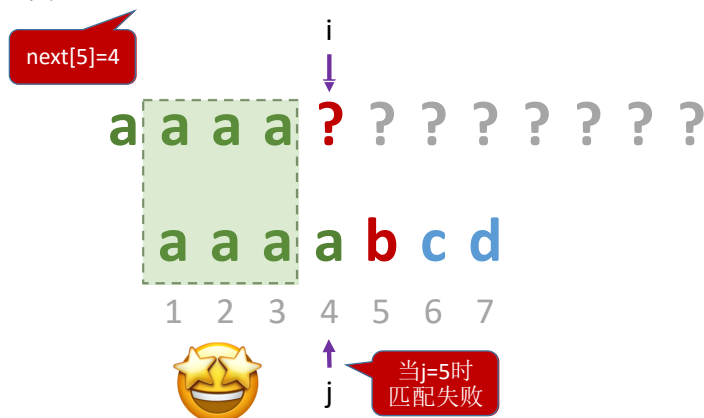
王道考研/CSKAOYAN.COM

12

观察: 求模式串的next数组

next 数组: 当模式串的第 j 个字符匹配失败时, 令模式串跳到 $\text{next}[j]$ 再继续匹配

模式串: 'aaaabcd'



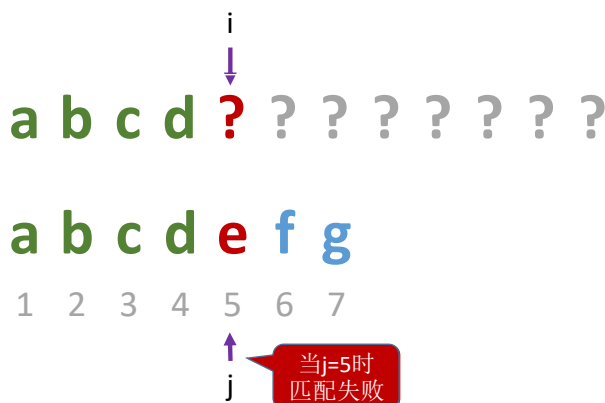
王道考研/CSKAOYAN.COM

13

观察: 求模式串的next数组

next 数组: 当模式串的第 j 个字符匹配失败时, 令模式串跳到 $\text{next}[j]$ 再继续匹配

模式串: 'abcdefg'



王道考研/CSKAOYAN.COM

14

观察：求模式串的next数组

next 数组：当模式串的第 j 个字符匹配失败时，令模式串跳到 next[j] 再继续匹配

模式串：'abcdefg'

a

b

c

d

?

?

?

?

?

?

?

?

1

2

3

4

5

6

7

a

b

c

d

e

f

g

</

观察：求模式串的next数组

next 数组：当模式串的第 j 个字符匹配失败时，令模式串跳到 next[j] 再继续匹配

模式串：'abcdefg'

i

↓

a b c d ? ? ? ? ? ? ? ?

a b c d e f g

1 2 3 4 5 6 7

🤔

↑

j

当j=5时
匹配失败

王道考研/CSKAOYAN.COM

17

观察：求模式串的next数组

next 数组：当模式串的第 j 个字符匹配失败时，令模式串跳到 next[j] 再继续匹配

模式串：'abcdefg'

i

↓

a b c d ? ? ? ? ? ? ? ?

a b c d e f g

1 2 3 4 5 6 7

🌟

↑

j

当j=5时
匹配失败

next[5]=1

王道考研/CSKAOYAN.COM

18

王道考研/CSKAOYAN.COM

9

观察：求模式串的next数组

next 数组：当模式串的第 j 个字符匹配失败时，令模式串跳到 next[j] 再继续匹配

模式串：'abcabd'

i

↓

?

?

?

?

?

?

?

?

?

a

b

c

a

b

d

1

2

3

4

5

6

↑

j

当j=1时
匹配失败

王道考研/CSKAOYAN.COM

19

观察：求模式串的next数组

next 数组：当模式串的第 j 个字符匹配失败时，令模式串跳到 next[j] 再继续匹配

模式串：'abcabd'

next[1]=0

i

↓

?

?

?

?

?

?

?

?

?

a

b

c

a

b

d

1

2

3

4

5

6

↑

j

当j=1时
匹配失败

王道考研/CSKAOYAN.COM

20

王道考研,cskaoyan.com

10

KMP算法代码

```
int Index_KMP(SSString S,SSString T,int next[]){
    int i=1, j=1;
    while(i<=S.length&& j<=T.length){
        if(j==0 || S.ch[i]==T.ch[j]){
            ++i;
            ++j;           //继续比较后继字符
        }
        else {
            j=next[j];     //模式串向右移动
        }
    }
    if(j>T.length)
        return i-T.length; //匹配成功
    else
        return 0;
}
```

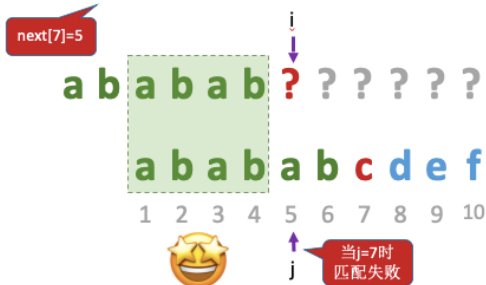
王道考研/CSKAOYAN.COM

21

总结: 求模式串的next数组

next 数组: 当模式串的第 j 个字符匹配失败时, 令模式串跳到 next[j] 再继续匹配

模式串: 'ababab c d e f'



模式串: 'abc d e f g'



串的前缀: 包含第一个字符, 且不包含最后一个字符的子串
 串的后缀: 包含最后一个字符, 且不包含第一个字符的子串

当第 j 个字符匹配失败, 由前 1~j-1 个字符组成的串记为 S, 则:
 $next[j] = S \text{ 的最长相等前后缀长度} + 1$
 特别地, $next[1] = 0$

王道考研/CSKAOYAN.COM

22

练习：求模式串的next数组

串的前缀：包含第一个字符，且不包含最后一个字符的子串
串的后缀：包含最后一个字符，且不包含第一个字符的子串
当第j个字符匹配失败，由前 1~j-1 个字符组成的串记为S，则：
next[j]= S的最长相等前后缀长度+1
特别地，next[1]=0

模式串：'ababaa'

序号j	1	2	3	4	5	6
模式串	a	b	a	b	a	a
next[j]						

王道考研/CSKAOYAN.COM

23

练习：求模式串的next数组

串的前缀：包含第一个字符，且不包含最后一个字符的子串
串的后缀：包含最后一个字符，且不包含第一个字符的子串
当第j个字符匹配失败，由前 1~j-1 个字符组成的串记为S，则：
next[j]= S的最长相等前后缀长度+1
特别地，next[1]=0

模式串：'ababaa'

序号j	1	2	3	4	5	6
模式串	a	b	a	b	a	a
next[j]	0	1	1	2	3	4

王道考研/CSKAOYAN.COM

24

练习：求模式串的next数组

串的前缀：包含第一个字符，且不包含最后一个字符的子串
串的后缀：包含最后一个字符，且不包含第一个字符的子串
当第j个字符匹配失败，由前 1~j-1 个字符组成的串记为S，则：
next[j]= S的最长相等前后缀长度+1
特别地，next[1]=0

模式串：'aaaab'

序号j	1	2	3	4	5
模式串	a	a	a	a	b
next[j]					

王道考研/CSKAOYAN.COM

25

练习：求模式串的next数组

串的前缀：包含第一个字符，且不包含最后一个字符的子串
串的后缀：包含最后一个字符，且不包含第一个字符的子串
当第j个字符匹配失败，由前 1~j-1 个字符组成的串记为S，则：
next[j]= S的最长相等前后缀长度+1
特别地，next[1]=0

模式串：'aaaab'

序号j	1	2	3	4	5
模式串	a	a	a	a	b
next[j]	0	1	2	3	4

王道考研/CSKAOYAN.COM

26

练习: 求模式串的next数组

串的前缀: 包含第一个字符, 且不包含最后一个字符的子串
 串的后缀: 包含最后一个字符, 且不包含第一个字符的子串
 当第j个字符匹配失败, 由前 1~j-1 个字符组成的串记为S, 则:
 next[j]=S的最长相等前后缀长度+1
 特别地, next[1]=0

$$next[j] = \begin{cases} 0 & \text{当 } j=1 \text{ 时} \\ \text{Max}\{k \mid 1 < k < j \text{ 且 } 'p_1 \cdots p_{k-1}' = 'p_{j-k+1} \cdots p_{j-1}'\} & \text{当此集合不空时} \\ 1 & \text{其他情况} \end{cases}$$

最长相等前后缀长度+1

前缀

后缀

前后缀不匹配

王道考研/CSKAOYAN.COM

27

KMP算法性能分析

```
//求模式串T的next数组
void get_next(SSString T,int next[]){
    int i=1, j=0;
    next[1]=0;
    while(i<T.length){
        if(j==0 || T.ch[i]==T.ch[j]){
            ++i; ++j;
            //若pi=pj, 则 next[j+1]=next[j]+1
            next[i]=j;
        }
        else
            //否则令j=next[j], 循环继续
            j=next[j];
    }
}
```



我已经放弃治疗了

KMP 算法平均时间复杂度: $O(n+m)$

```
//KMP算法
int Index_KMP(SSString S,SSString T){
    int i=1, j=1;
    int next[T.length+1];
    get_next(T, next); //求模式串的next数组
    while(i<=S.length&& j<=T.length){
        if(j==0 || S.ch[i]==T.ch[j]){
            ++i;
            ++j; //继续比较后继字符
        }
        else
            j=next[j]; //模式串向右移动
    }
    if(j>T.length)
        return i-T.length; //匹配成功
    else
        return 0;
}
```

$O(m)$

$O(n)$

王道考研/CSKAOYAN.COM

28

知识回顾与重要考点

朴素模式匹配算法的缺点: 当某些子串与模式串能部分匹配时, 主串的扫描指针 i 经常回溯, 导致时间开销增加。最坏时间复杂度 $O(nm)$

KMP算法: 当子串和模式串不匹配时, 主串指针 i 不回溯, 模式串指针 $j = \text{next}[j]$
算法平均时间复杂度: $O(n+m)$

next数组手算方法: 当第 j 个字符匹配失败, 由前 $1 \sim j-1$ 个字符组成的串记为 S , 则:
 $\text{next}[j] = S$ 的最长相等前后缀长度 + 1
特别地, $\text{next}[1] = 0$

哼 不过如此嘛



如果不会经常出现子串与模式串部分匹配问题, 那么 KMP 算法也没屁多少

王道考研/CSKAOYAN.COM

本节内容

KMP算法优化
——nextval数组

王道考研/CSKAOYAN.COM

1

例1：KMP算法存在的问题

↓

g

1

→

o

2

→

o

3

↓

g

4

→

g

5

→

g

6

→

o

7

→

o

8

→

g

9

→

l

10

→

e

11

→

x

12

→

x

13

→

x

14

↑

g

1

→

o

2

→

o

3

↑

g

4

→

l

5

→

e

6

↑

↓

j

i

序号j	1	2	3	4	5	6
模式串	g	o	o	g	l	e
next[j]	0	1	1	1	2	1

王道考研/CSKAOYAN.COM

2

例1：KMP算法存在的问题

g

o

o

i

!

g

g

o

o

g

l

e

x

x

x

1

2

3

4

5

6

7

8

9

10

11

12

13

14

g

o

o

g

l

e

1

2

3

4

5

6

j

序号j	1	2	3	4	5	6
模式串	g	o	o	g	l	e
next[j]	0	1	1	1	2	1

王道考研/CSKAOYAN.COM

3

例1：KMP算法存在的问题

g

o

o

i

!

g

g

o

o

g

l

e

x

x

x

1

2

3

4

5

6

7

8

9

10

11

12

13

14

g

o

o

g

l

e

1

2

3

4

5

6

j

序号j	1	2	3	4	5	6
模式串	g	o	o	g	l	e
next[j]	0	1	1	1	2	1

王道考研/CSKAOYAN.COM

4

例1：KMP算法存在的问题

序号j	1	2	3	4	5	6
模式串	g	o	o	g	l	e
next[j]	0	1	1	1	2	1

王道考研/CSKAOYAN.COM

5

例1：KMP算法存在的问题

序号j	1	2	3	4	5	6
模式串	g	o	o	g	l	e
next[j]	0	1	1	1	2	1

王道考研/CSKAOYAN.COM

6

KMP算法优化

↓

→

↓

g

o

o

g

g

o

o

g

l

e

x

x

x

1

2

3

4

5

6

7

8

9

10

11

12

13

14

↑

→

↑

g

o

o

g

l

e

1

2

3

4

5

6

↑

→

↑

j

多进行了一次无意义的对比

不要慌，问题不大

序号j	1	2	3	4	5	6
模式串	g	o	o	g	l	e
next[j]	0	1	1	1	2	1
nextval[j]	0	1	1	0	2	1

对next数组的优化

王道考研/CSKAOYAN.COM

7

KMP算法优化

↓

→

↓

g

o

o

g

g

o

o

g

l

e

x

x

x

1

2

3

4

5

6

7

8

9

10

11

12

13

14

↑

→

↑

g

o

o

g

l

e

1

2

3

4

5

6

↑

→

↑

j

多进行了一次无意义的对比

不要慌，问题不大

序号j	1	2	3	4	5	6
模式串	g	o	o	g	l	e
next[j]	0	1	1	1	2	1
nextval[j]	0	1	1	0	2	1

对next数组的优化

王道考研/CSKAOYAN.COM

8

例2：KMP算法存在的问题

序号j	1	2	3	4	5
模式串	a	a	a	a	b
next[j]	0	1	2	3	4

王道考研/CSKAOYAN.COM

9

例2：KMP算法存在的问题

序号j	1	2	3	4	5
模式串	a	a	a	a	b
next[j]	0	1	2	3	4

王道考研/CSKAOYAN.COM

10

例2：KMP算法存在的问题

i

a

a

a

c

a

a

a

a

b

e

x

x

x

x

1

2

3

4

5

6

7

8

9

10

11

12

13

14

a

a

a

a

b

1

2

3

4

5

j

序号j	1	2	3	4	5
模式串	a	a	a	a	b
next[j]	0	1	2	3	4

王道考研/CSKAOYAN.COM

11

例2：KMP算法存在的问题

i

a

a

a

c

a

a

a

a

b

e

x

x

x

x

1

2

3

4

5

6

7

8

9

10

11

12

13

14

a

a

a

a

b

1

2

3

4

5

j

序号j	1	2	3	4	5
模式串	a	a	a	a	b
next[j]	0	1	2	3	4

王道考研/CSKAOYAN.COM

12

例2：KMP算法存在的问题

序号j	1	2	3	4	5
模式串	a	a	a	a	b
next[j]	0	1	2	3	4

王道考研/CSKAOYAN.COM

13

KMP算法优化

序号j	1	2	3	4	5
模式串	a	a	a	a	b
next[j]	0	1	2	3	4
nextval[j]	0	0	0	0	4

对next数组的优化

王道考研/CSKAOYAN.COM

14

KMP算法优化

序号j	1	2	3	4	5
模式串	a	a	a	a	b
next[j]	0	1	2	3	4
nextval[j]	0	0	0	0	4

王道考研/CSKAOYAN.COM

15

KMP算法考点总结

KMP算法: 当子串和模式串不匹配时, 主串指针 i 不回溯, 模式串指针 $j = \text{next}[j]$
 算法平均时间复杂度: $O(n+m)$

next数组手算方法: 当第 j 个字符匹配失败, 由前 $1 \sim j-1$ 个字符组成的串记为 S , 则:
 $\text{next}[j] = S$ 的最长相等前后缀长度 + 1
 特别地, $\text{next}[1] = 0$

nextval数组的求法:
 先算出next数组
 先令 $\text{nextval}[1] = 0$
 for (int j=2; j<=T.length; j++) {
 if (T.ch[next[j]] == T.ch[j])
 $\text{nextval}[j] = \text{nextval}[\text{next}[j]]$;
 else
 $\text{nextval}[j] = \text{next}[j]$;
 }

序号j	1	2	3	4	5	6
模式串	a	b	a	b	a	a
next[j]	0	1	1	2	3	4
nextval[j]	0	1	0	1	0	4

KMP算法优化: 当子串和模式串不匹配时 $j = \text{nextval}[j]$;

王道考研/CSKAOYAN.COM

16